

Trabajo Final Integrador: Sistema “EcoGrid” – Micro-Redes Energéticas Inteligentes

- Asignatura: Programación Orientada a Objetos
- Año: 2026
- Modalidad: Grupal (máximo 3 integrantes)

1. Objetivos

Al finalizar este Trabajo Final, los alumnos serán capaces de:

- Diseñar e implementar una arquitectura de software en capas aplicando Programación Orientada a Objetos en C++17, con herencia, polimorfismo, encapsulamiento y manejo automático de memoria (RAII, smart pointers).
- Modelar datos relacionales normalizados, crear esquemas en Oracle SQL y desarrollar lógica de negocio del lado del servidor mediante triggers y procedimientos almacenados.
- Resolver problemas algorítmicos con estructuras de datos eficientes (subasta doble continua) y gestionar transaccionalidad entre aplicación y base de datos.
- Conectar aplicaciones C++ con Oracle utilizando la librería SOCI, manejando excepciones, cursores y conversiones de tipos.
- Documentar diseño (UML, DER) y decisiones de implementación, y exponer oralmente el trabajo realizado.

2. Contexto del Problema

En el año 2026, las ciudades inteligentes permiten que los ciudadanos no solo consuman energía, sino que también la produzcan (mediante paneles solares, mini-eólicos, etc.) y la vendan a sus vecinos en una micro-red local. El sistema EcoGrid debe gestionar los nodos de la red, simular la producción/consumo en tiempo real, registrar las transacciones de energía y calcular las tarifas dinámicas según la oferta y la demanda.

3. Descripción Funcional del Sistema

3.1 Actores del Sistema (Nodos de la Red)

NodoConsumidor: Hogares o industrias que solo compran energía. Tienen un perfil de consumo (Residencial, Comercial, Industrial) que define su demanda típica.

NodoProsumidor: Produce y consume energía (ej. paneles solares). Calcula su excedente neto en cada ciclo y puede vender a la red.

NodoAlmacenamiento (Batería Comunitaria): Absorbe el excedente global de energía al final de cada tick si no se pudo vender, y lo reinyecta como oferta de venta en el siguiente tick a un precio de referencia definido (ver reglas). Actúa como un vendedor especial que solo vende cuando tiene carga acumulada.

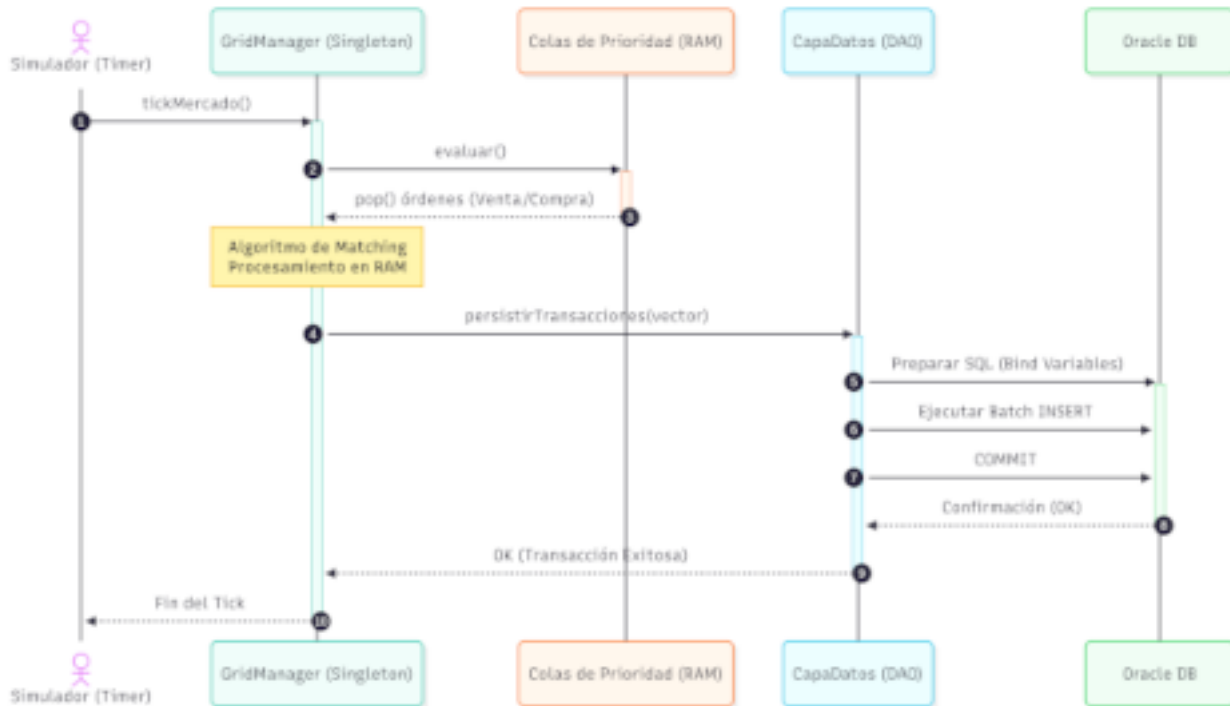


diagrama de secuencia

3.2 Flujo de un Tick de Simulación

Cada tick representa una hora simulada. El sistema opera en ciclos discretos de la siguiente manera:

Carga de órdenes desde archivo CSV:

Se lee un archivo ofertas_HH.csv (donde HH es la hora del día, 00 a 23) que contiene las órdenes de compra y venta de los nodos para ese período. Las órdenes se insertan en las estructuras de memoria del libro de órdenes.

Ejecución del Algoritmo de Subasta (GridManager):

Se procesa el libro de órdenes en RAM utilizando el mecanismo de Subasta Doble Continua (descrito en detalle en la Sección 6). Se generan transacciones de energía entre compradores y vendedores mientras exista compatibilidad de precios.

Intervención de la Batería Comunitaria (opcional al principio, obligatorio en versión final):

Una vez agotado el matching normal, si hay excedente no vendido, el GridManager lo transfiere a la batería comunitaria (si existe), registrando una transacción especial donde el nodo batería “compra” al precio base de excedente (ver Sección 3.3). En el tick siguiente, la batería oferta ese excedente como órdenes de venta con precio igual al precio base horario.

Persistencia Transaccional:

Todas las transacciones calculadas en ese tick se envían a Oracle en un único bloque transaccional (BEGIN...COMMIT). Si alguna inserción falla (por violación de reglas o error de conexión), se ejecuta ROLLBACK y el GridManager es notificado para reintento o registro de error.

Actualización de Saldos y Lecturas:

Luego del COMMIT exitoso, un procedimiento almacenado en Oracle actualiza los saldos de cuenta de los nodos y registra las lecturas de producción/consumo en la tabla LECTURAS_HISTORICAS.

3.3 Reglas de Negocio y Precios

Precio de transacción (clearing): Se calcula como el promedio entre el precio máximo ofrecido por el comprador y el precio mínimo pedido por el vendedor:

$$\text{precio_clearing} = (\text{precio_compra_max} + \text{precio_venta_min}) / 2.0$$

Tarifas dinámicas: El precio base del kWh por hora se consulta de la tabla CONFIG_TARIFAS. Las órdenes de compra/venta no pueden tener precios menores a cero.

Saldo de cuenta: Cada nodo tiene un saldo (en créditos) que se actualiza con cada transacción. Un nodo no puede emitir una orden de compra si su saldo actual es insuficiente para cubrir el costo estimado de la operación (validado por trigger en BD).

Excedentes a batería: Si al final del tick hay vendedores con energía sobrante que no encontró comprador, esa energía se vende automáticamente a la batería comunitaria al precio base horario (definido en CONFIG_TARIFAS para esa hora). La batería asume ese costo y lo descuenta de su propio saldo (ilimitado para simplificar).

Energía de batería: En un tick, la batería oferta su energía acumulada como órdenes de venta al precio base horario (se insertan al inicio del libro de ventas antes de procesar las del CSV).

4. Requerimientos Técnicos

4.1 Modelo de Clases (C++)

El diseño debe demostrar uso sólido de POO. Se debe utilizar herencia, polimorfismo, encapsulamiento.

Jerarquía de Clases Principal:

NodoRed (abstracta): - Atributos: int id, std::string ubicacion, double balanceEnergia (kW actual), double saldoCuenta (créditos). - Métodos virtuales puros: virtual double calcularExcedente() = 0;

NodoConsumidor (hereda de NodoRed): - Atributo adicional: PerfilConsumo perfil (enum Residencial, Comercial, Industrial). - calcularExcedente() retorna siempre un valor negativo o cero (demanda).

NodoProsumidor (hereda de NodoRed): - Atributos adicionales: double produccion, double consumo. - calcularExcedente() retorna produccion - consumo (positivo si hay excedente).

NodoAlmacenamiento (hereda de NodoRed): - Atributo adicional: double cargaActual (kWh). - Sobrescribe métodos para absorber y liberar energía.

TransaccionEnergia (entidad): - int idVendedor, idComprador; double kwh; double precio; std::chrono::system_clock::time_point timestamp;

Orden (struct): - int idOrden; bool esCompra; // true=compra, false=venta - int idNodo; double precio; double kwh; uint64_t secuencia; // timestamp o contador FIFO

GridManager: Contiene el libro de órdenes y ejecuta el matching. Método principal void procesarTick(const std::vector& ofertasCSV);

CapaDatos: Se conecta a Oracle usando SOCI. Métodos: void persistirTransacciones(const std::vector& trans); etc.

4.2 Esquema de Base de Datos (Oracle)

Se debe entregar un script SQL que cree al menos las siguientes tablas:

Tabla NODOS

```
CREATE TABLE NODOS (  
  id_nodo NUMBER PRIMARY KEY,  
  ubicacion VARCHAR2(100) NOT NULL,  
  tipo VARCHAR2(20) CHECK (tipo IN ('Consumidor','Prosumidor','Bateria')),  
  saldo_cuenta NUMBER DEFAULT 0 CHECK (saldo_cuenta >= 0), -- créditos
```

```

    perfil_consumo VARCHAR2(20) -- solo para Consumidores
);

```

Tabla LECTURAS_HISTORICAS

```

CREATE TABLE LECTURAS_HISTORICAS (
    id_lectura NUMBER GENERATED BY DEFAULT AS IDENTITY PRIMARY KEY,
    id_nodo NUMBER NOT NULL,
    tick_hora DATE NOT NULL, -- timestamp simulado
    produccion_kwh NUMBER DEFAULT 0,
    consumo_kwh NUMBER DEFAULT 0,
    excedente_neto NUMBER,
    FOREIGN KEY (id_nodo) REFERENCES NODOS(id_nodo)
);

```

Tabla TRANSACCIONES

```

CREATE TABLE TRANSACCIONES (
    id_transaccion NUMBER GENERATED BY DEFAULT AS IDENTITY PRIMARY KEY,
    id_vendedor NUMBER NOT NULL,
    id_comprador NUMBER NOT NULL,
    kwh NUMBER NOT NULL,
    precio_unitario NUMBER NOT NULL,
    fecha_transaccion TIMESTAMP DEFAULT CURRENT_TIMESTAMP,
    FOREIGN KEY (id_vendedor) REFERENCES NODOS(id_nodo),
    FOREIGN KEY (id_comprador) REFERENCES NODOS(id_nodo),
    CONSTRAINT chk_kwh_pos CHECK (kwh > 0),
    CONSTRAINT chk_precio_pos CHECK (precio_unitario > 0)
);

```

Tabla CONFIG_TARIFAS

```

CREATE TABLE CONFIG_TARIFAS (
    hora NUMBER(2) PRIMARY KEY CHECK (hora BETWEEN 0 AND 23),
    precio_base_kwh NUMBER NOT NULL
);

```

Disparador (Trigger):

```

CREATE OR REPLACE TRIGGER trg_validar_saldo
BEFORE INSERT ON TRANSACCIONES
FOR EACH ROW
DECLARE
    v_saldo_comprador NUMBER;
BEGIN

```

```

SELECT saldo_cuenta INTO v_saldo_comprador
FROM NODOS WHERE id_nodo = :NEW.id_comprador;

IF v_saldo_comprador < (:NEW.kwh * :NEW.precio_unitario) THEN
    RAISE_APPLICATION_ERROR(-20001, 'Saldo insuficiente para realizar
la compra');
END IF;
END;

```

Procedimiento almacenado obligatorio:

actualizar_saldo_y_lecturas(p_id_nodo, p_kwh, p_precio, p_tipo_operacion)
que actualice saldo_cuenta en NODOS e inserte en LECTURAS_HISTORICAS.
Se ejecutará después de cada transacción confirmada.

4.3 Conexión C++ a Oracle (CapaDatos con SOCI)

La CapaDatos debe usar la librería SOCI para abstraer sentencias SQL.

Todas las operaciones de un tick deben ejecutarse en una transacción explícita:
soci::transaction tr(sql); ... tr.commit(); con manejo de excepciones y rollback en
caso de error.

La conexión se configurará mediante un archivo config.ini con parámetros de
conexión.

Los tipos de datos de Oracle (NUMBER, VARCHAR2, TIMESTAMP) deben
mapearse correctamente a tipos C++ (int, std::string, std::tm o std::chrono).

5. Especificación de la Subasta Doble Continua

5.1 Estructura del Libro de Órdenes

Se implementará utilizando std::map<double, std::queue> para las ofertas de
compra (bidMap) y de venta (askMap):

bidMap: Mapa ordenado de mayor a menor (usando std::greater en la com-
paración). La clave es el precio, el valor es una cola FIFO de órdenes a ese
precio.

askMap: Mapa ordenado de menor a mayor (orden por defecto). Clave precio,
valor cola FIFO.

Esto garantiza la prioridad precio-tiempo: el mejor precio se obtiene con begin()
de cada mapa, y dentro de un precio se despacha la orden más antigua.

Ejemplo de declaraciones:

```
std::map<double, std::queue<Orden>, std::greater<double>> bidMap;
std::map<double, std::queue<Orden>> askMap;
```

5.2 Algoritmo de Matching (pseudocódigo)

```
mientras (bidMap no vacío Y askMap no vacío) {
    mejorBid = bidMap.begin(); // precio más alto de compra
    mejorAsk = askMap.begin(); // precio más bajo de venta

    si (mejorBid->first >= mejorAsk->first) { // precios compatibles
        ordenCompra = mejorBid->second.front();
        ordenVenta = mejorAsk->second.front();

        energia = min(ordenCompra.kwh, ordenVenta.kwh);
        precio = (ordenCompra.precio + ordenVenta.precio) / 2.0;

        registrarTransaccion(ordenVenta.idNodo, ordenCompra.idNodo,
            energia, precio);

        // Actualizar remanentes
        ordenCompra.kwh -= energia;
        ordenVenta.kwh -= energia;

        if (ordenCompra.kwh > umbral) // umbral = 0.001
            mejorBid->second.push(ordenCompra); // vuelve al final de
la cola (mismo precio)
        else
            mejorBid->second.pop(); // orden completada

        if (ordenVenta.kwh > umbral)
            mejorAsk->second.push(ordenVenta);
        else
            mejorAsk->second.pop();

        // Limpiar colas vacías en el mapa
        if (mejorBid->second.empty()) bidMap.erase(mejorBid);
        if (mejorAsk->second.empty()) askMap.erase(mejorAsk);
    }
    else {
        break; // no hay más cruces posibles
    }
}
```

5.3 Ejemplo Numérico Concreto

Datos de entrada (CSV):

```
id_orden lado id_nodo kwh precio 101 venta 5 10 2.5 102 venta 3 5 2.8 201 compra
7 8 3.0 202 compra 9 6 2.4
```

Ejecución esperada:

mejorBid = 201 (precio 3.0), mejorAsk = 101 (precio 2.5). Compatible ($3.0 \geq 2.5$). Transfiere $\min(8,10)=8$ kWh a precio $(3.0+2.5)/2 = 2.75$. Quedan: orden 201 completada (se elimina), orden 101 con 2 kWh remanentes (vuelve a cola en askMap).

mejorBid = 202 (precio 2.4), mejorAsk = 101 (precio 2.5). No compatible ($2.4 < 2.5$). Fin del matching.

Resultado: Una transacción registrada. Excedente de venta 2 kWh irán a batería si existe, al precio base horario.

6. Entrada y Salida del Sistema

6.1 Archivo de Entrada CSV

Durante cada tick, el programa cargará un archivo llamado ofertas_HH.csv (ej. ofertas_12.csv) ubicado en la carpeta ./datos/. Formato:

```
id_orden, lado, id_nodo, kwh, precio 101,venta,5,10,2.5 102,venta,3,5,2.8 201,compra,7,8,3.0 202,compra,9,6,2.4
```

 lado indica “compra” o “venta”.

Los nodos deben existir previamente en la base de datos.

La simulación puede correr todos los ticks de un día (00 a 23) secuencialmente, leyendo los archivos uno por uno.

6.2 Salidas Esperadas

Consola: Log de cada transacción realizada (nodos, kWh, precio) y resumen por tick.

Base de datos: Inserción en las tablas TRANSACCIONES y LECTURAS_HISTORICAS, y actualización de saldo_cuenta en NODOS.

7. Entregables y Organización

El proyecto se entregara en el aula virtual con una estructura similar a esta:

- ecogrid

- /src
 - ★ /ecogrid.h
 - ★ /util (archivos de conf.)
 - ★ main.cpp
- /sql
 - ★ crear_esquema.sql (tablas, triggers, procedimientos, datos semilla)
- /docs
 - ★ UML_diagrama_clases.png/.pdf
 - ★ DER_diagrama_entidad_relacion.png/.pdf
 - ★ Informe_Tecnico.pdf
- /datos
 - ★ ofertas_00.csv
 - ★ ofertas_01.csv
 - ★ ... (24 archivos)

Documentos:

Diagrama de Clases UML: Debe mostrar herencias, relaciones (agregación/composición), clases de soporte y patrones utilizados.

Diagrama Entidad-Relación (DER): Esquema completo de tablas, claves foráneas y restricciones.

Informe Técnico: Decisiones de diseño, desafíos encontrados, distribución de tareas y conclusiones. Máximo 5 páginas.

8. Cronograma y Evaluación

8.1 Hitos

Recomendamos la siguiente secuencia de trabajo:

Hito Descripción Peso 1 Diagrama UML y DER + Script SQL con datos semilla y trigger funcionando. 25% 2 Implementación del motor de subasta en C++ (sin BD) con lectura de CSV y salida por consola. 35% 3 Integración completa con Oracle, transaccionalidad, batería y defensa oral. 40%

Pueden ir haciendo entregas parciales del trabajo, muestras de avances y consultas

8.2 Rúbrica de Evaluación

Correctitud funcional (40%): El matching produce transacciones correctas según el ejemplo y pruebas; persistencia sin errores; respeto de reglas de negocio.

Calidad del código (25%): Uso adecuado de POO, encapsulamiento, organización en capas, código limpio.

Base de datos (20%): Diseño relacional normalizado, triggers y procedimientos funcionales, transacciones ACID.

Documentación y defensa (15%): Diagramas correctos, informe claro, exposición oral coherente y demostración funcional.

Condiciones de aprobación:

Todas las transacciones deben persistir sin pérdidas.

El trigger de saldo insuficiente debe dispararse en una prueba provista. Podría validarse por software si lo prefieren

9. Recursos y Configuración del Entorno

- Compilador: GCC 11+ o Clang 13+ con C++17.
- Librería SOCI: Se dejaron las instrucciones en el aula virtual.
- Oracle Database: provista por la cátedra de base de datos.
- Herramientas de modelado: LibreOffice Draw, Dia, PlantUML o cualquier software para UML y DER.

Casos de Uso de Ejemplo y Validación

Esta sección presenta escenarios concretos que el sistema debe manejar correctamente. Se recomienda utilizarlos como pruebas de aceptación durante el desarrollo.

En todos los casos se asume que la tabla CONFIG_TARIFAS está poblada con los siguientes precios base por hora (fracción):

Hora	Precio base kWh
10	1.00
12	1.50
14	1.00
15	1.20
18	2.00

Caso 1: Subasta básica exitosa (Tick 10:00)

Estado inicial de la BD:

Nodo 1: Consumidor Residencial, saldo = 100.00 créditos.

Nodo 2: Prosumidor, saldo = 50.00 créditos.

Archivo ofertas_10.csv:

```
id_orden,lado,id_nodo,kwh,precio
1001,venta,2,5,2.0
2001,compra,1,5,2.5
```

Procesamiento esperado:

bidMap mejor compra: precio 2.5 (2001). **askMap mejor venta:** precio 2.0 (1001).

Compatibles (2.5 2.0). Energía transferida: $\min(5,5) = 5$ kWh. Precio clearing: $(2.5+2.0)/2 = 2.25$.

Ambas órdenes quedan completadas (remanente 0), se eliminan.

No hay excedentes. Batería no actúa.

Transacción generada (memoria):

idVendedor=2, idComprador=1, kwh=5.0, precio=2.25

Persistencia y estado final BD esperado:

Se inserta una fila en TRANSACCIONES con esos datos.

Se actualizan saldos en NODOS mediante el procedimiento almacenado:

Nodo 1: $100 - (5 * 2.25) = 88.75$

Nodo 2: $50 + (5 * 2.25) = 61.25$

Se insertan lecturas históricas para ambos nodos reflejando la transacción (producción/consumo neto).

El trigger de saldo no se dispara porque el saldo del comprador era suficiente.

Caso 2: Prioridad precio-tiempo y múltiples órdenes (Tick 12:00)

Estado inicial (nodos existentes con saldo alto):

Nodo 10: Consumidor Industrial, saldo = 500.

Nodo 11: Consumidor Comercial, saldo = 200.

Nodo 20: Prosumidor, saldo = 100.

Nodo 21: Prosumidor, saldo = 80.

Archivo ofertas_12.csv:

```
id_orden,lado,id_nodo,kwh,precio
3001,venta,20,10,2.0
3002,venta,21,8,2.0
3003,venta,22,5,2.5
4001,compra,10,12,3.0
4002,compra,11,6,2.2
```

(Nota: se añade nodo 22 Prosumidor con saldo 50, existe en BD; en el CSV aparecen órdenes de nodos existentes).

Ejecución del algoritmo:

bidMap inicial:

precio 3.0: cola [4001 (12 kWh)]
precio 2.2: cola [4002 (6 kWh)]

askMap inicial:

precio 2.0: cola [3001 (10 kWh), 3002 (8 kWh)] (orden de llegada: 3001 primero)
precio 2.5: cola [3003 (5 kWh)]

Paso 1:

mejorBid = 3.0 (4001), mejorAsk = 2.0 (3001). Compatible.

Transfiere $\min(12,10) = 10$ kWh. Precio = $(3.0+2.0)/2 = 2.5$.

3001 completada (se elimina). 4001 queda con 2 kWh remanentes, se reinserta en bidMap (sigue al final de la cola en 3.0).

Estado tras paso 1:

bidMap: 3.0 $\rightarrow$ [4001 (2 kWh)]; 2.2 $\rightarrow$ [4002 (6 kWh)]

askMap: 2.0 $\rightarrow$ [3002 (8 kWh)]; 2.5 $\rightarrow$ [3003 (5 kWh)]

Paso 2:

mejorBid = 3.0 (4001, 2 kWh), mejorAsk = 2.0 (3002, 8 kWh). Compatible.

Transfiere $\min(2,8) = 2$ kWh. Precio = $(3.0+2.0)/2 = 2.5$.

4001 completada (se elimina). 3002 queda con 6 kWh, se reinserta en askMap.

Estado:

bidMap: $2.2 \rightarrow [4002 \text{ (6 kWh)}]$

askMap: $2.0 \rightarrow [3002 \text{ (6 kWh)}]$; $2.5 \rightarrow [3003 \text{ (5 kWh)}]$

Paso 3:

mejorBid = 2.2 (4002), mejorAsk = 2.0 (3002). Compatible (2.2 2.0).

Transfiere $\min(6,6) = 6$ kWh. Precio = $(2.2+2.0)/2 = 2.1$.

Ambas completadas y eliminadas.

Estado final:

bidMap: vacío.

askMap: $2.5 \rightarrow [3003 \text{ (5 kWh)}]$ (queda sin vender).

Transacciones generadas en memoria:

(vendedor=20, comprador=10, 10 kWh, 2.5)

(vendedor=21, comprador=10, 2 kWh, 2.5)

(vendedor=21, comprador=11, 6 kWh, 2.1)

Excedente:

La orden 3003 (nodo 22, 5 kWh a 2.5) no encontró comprador. Como existe una batería comunitaria (nodo 99, configurado en BD con saldo ilimitado), el GridManager debe transferir ese excedente a la batería al precio base horario de las 12:00 (1.50). Se genera una transacción adicional:

(vendedor=22, comprador=99, 5 kWh, 1.50)

Actualización de saldos esperada (luego de COMMIT):

Nodo 10: $500 - (10*2.5 + 2*2.5) = 500 - 30 = 470$

Nodo 11: $200 - (6*2.1) = 200 - 12.6 = 187.4$

Nodo 20: $100 + 25 = 125$

Nodo 21: $80 + (2 \cdot 2.5 + 6 \cdot 2.1) = 80 + (5 + 12.6) = 97.6$

Nodo 22: $50 + (5 \cdot 1.5) = 57.5$

Nodo 99 (Batería): saldo se descuenta ilimitado (no controlamos saldo de batería, solo carga). La carga de la batería aumenta en 5 kWh.

Caso 3: Trigger de saldo insuficiente y manejo de error (Tick 18:00)

Estado inicial:

Nodo 30: Consumidor Residencial, saldo = 15.00 créditos.

Nodo 40: Prosumidor, saldo = 60.

Archivo ofertas_18.csv:

```
id_orden,lado,id_nodo,kwh,precio
5001,venta,40,10,2.0
6001,compra,30,10,2.5
```

Ejecución en memoria:

La subasta empareja sin problemas: 10 kWh a precio $(2.5+2.0)/2 = 2.25$. Costo total = 22.50.

La transacción se construye en memoria y se pasa a CapaDatos::persistirTransacciones().

Comportamiento esperado en la base de datos:

Al intentar insertar en TRANSACCIONES, el trigger trg_validar_saldo verifica el saldo del comprador (nodo 30): $15.00 < 22.50$.

El trigger lanza RAISE_APPLICATION_ERROR(-20001, 'Saldo insuficiente...').

La inserción falla. La CapaDatos captura la excepción de SOCI (soci::soci_error o similar) y ejecuta ROLLBACK.

Ninguna transacción de este tick se persiste (por atomicidad del bloque). El GridManager debe registrar un mensaje de error en el log indicando que el tick 18:00 fue rechazado por saldo insuficiente del nodo 30.

Los saldos en BD permanecen inalterados: nodo 30 sigue con 15.00, nodo 40 con 60.00.

Validación:

El programa no debe abortar; debe continuar con el siguiente tick (19:00). La transacción fallida no puede aparecer en la tabla TRANSACCIONES.

Caso 4: Integración completa con batería (Ticks 14:00 y 15:00)

Estado inicial:

Nodo 50: Prosumidor (saldo 100).

Nodo 51: Consumidor (saldo 200).

Nodo 99: Batería comunitaria (carga actual = 0 kWh; no tiene saldo controlable).

Precio base: 14:00 = 1.00, 15:00 = 1.20.

Tick 14:00 – Archivo ofertas_14.csv:

```
id_orden,lado,id_nodo,kwh,precio
7001,venta,50,8,2.0
8001,compra,51,3,1.8
```

Mejor venta: 2.0 (8 kWh). Mejor compra: 1.8 (3 kWh). No compatible ($1.8 < 2.0$).

Sin transacciones de mercado. Excedente del vendedor: 8 kWh.

Se transfiere automáticamente a la batería (nodo 99) al precio base 1.00.

Transacción: (vendedor=50, comprador=99, 8 kWh, 1.00).

Saldo nodo 50: $100 + 8 = 108$. Carga batería: $0 + 8 = 8$ kWh.

Tick 15:00 – Archivo ofertas_15.csv:

```
id_orden,lado,id_nodo,kwh,precio
9001,compra,51,10,1.5
```

(No hay ventas de prosumidores en este tick).

Antes de procesar el CSV, la batería genera su propia orden de venta:

Como tiene carga 8 kWh y el precio base horario es 1.20, inserta en el askMap: { id_orden: 0 (especial), lado: venta, id_nodo: 99, kwh: 8, precio: 1.20 }.

Luego se carga el CSV: la compra de 10 kWh a 1.5.

Matching en tick 15:00:

mejorBid = 1.5 (nodo 51, 10 kWh), mejorAsk = 1.20 (batería, 8 kWh).
Compatible.

Transfiere $\min(10,8) = 8$ kWh. Precio = $(1.5+1.2)/2 = 1.35$.

Transacción: (vendedor=99, comprador=51, 8 kWh, 1.35).

Remanente compra: 2 kWh a 1.5, pero no hay más vendedores. Esa orden sobra (no satisfecha, podría quedarse en libro para próximo tick o descartarse según política; en este TP, al no venderse, se descarta y se registra como demanda insatisfecha en log).

Actualización final esperada:

Saldo nodo 51: $200 - (8 * 1.35) = 200 - 10.8 = 189.2$.

Carga de batería: $8 - 8 = 0$ kWh.

Nodo 50 no participó en tick 15:00, saldo se mantiene en 108.

Se registra lectura de batería: inyectó 8 kWh.